IBM Certified Quantum Computation

using Qiskit v2.X Developer — Associate

Exam C1000-179

Section 1: Perform Quantum Operations

Complete Study Notes | 16% of Exam | ~11 Questions

Topics covered in this document:

#	Topic	Key focus
1	Single-qubit gates (Pauli, H, S, T)	Matrices, effects, eigenvalues
2	Rotation gates (Rx, Ry, Rz)	Bloch sphere, angle-probability link
3	Multi-qubit gates (CX, CZ, CCX, SWAP)	Truth tables, Qiskit syntax
4	The Pauli class & tensor products	qiskit.quantum_info, to_matrix()
5	Gate identities & relations	$H \cdot X \cdot H = Z$, $T^2 = S$, involutions
6	Statevectors & measurement	Amplitudes, probabilities, Qiskit
7	QuantumRegister & ClassicalRegister	Circuit construction, if_test
8	Exam tips & common traps	What the exam actually tests

1. Single-Qubit Gates

A single-qubit gate is a 2×2 unitary matrix. Applying a gate means multiplying that matrix by the qubit's statevector. All single-qubit gates are reversible, unitary ($U^\dagger U = I$), and preserve the norm of the statevector.

1.1 The Bloch Sphere Mental Model

Every single-qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ maps to a point on the unit sphere. The north pole is $|0\rangle$, the south pole is $|1\rangle$. The equator holds equal-superposition states like $|+\rangle$ and $|-\rangle$. Each Pauli gate is a 180° rotation around one of the three Cartesian axes. Hadamard is a 180° rotation around the diagonal axis halfway between X and Z.

1.2 The Pauli Gates

Pauli-I (Identity)

Matrix:

1	0
0	1

I = [[1,0],[0,1]]

Qiskit: `qc.id(0)`

Effect: Does nothing. Leaves every state unchanged.

- $I|0\rangle = |0\rangle$, $I|1\rangle = |1\rangle$
- Used as a placeholder in circuit diagrams
- $I^2 = I$ (trivially)
- **Hermitian:** Yes. Unitary: Yes.

Pauli-X (NOT gate / bit flip)

Matrix:

0	1
1	0

X = [[0,1],[1,0]]

Qiskit: `qc.x(0)`

Effect: Swaps $|0\rangle$ and $|1\rangle$. Quantum NOT gate.

- $X|0\rangle = |1\rangle$
- $X|1\rangle = |0\rangle$
- $X|+\rangle = |+\rangle$ (eigenstate, eigenvalue +1)
- $X|-\rangle = -|-\rangle$ (eigenstate, eigenvalue −1)
- $X^2 = I$ (self-inverse / involution)
- $H \cdot X \cdot H = Z$ (key identity)
- **Rotation:** 180° around x-axis on Bloch sphere

Pauli-Y

Matrix:

0	−i

Effect: Bit flip + phase flip. $Y = iXZ$.

- $Y|0\rangle = i|1\rangle$
- $Y|1\rangle = -i|0\rangle$

i	0

Y = [[0,−i],[i,0]]

Qiskit: `qc.y(0)`

- $Y^2 = I$ (involution)
- *The i is a global phase* when applied to a basis state
- **Rotation:** 180° around y-axis on Bloch sphere
- Eigenstates: $|i\rangle$ and $|-i\rangle$ (y-axis poles)

Pauli-Z (phase flip)

Matrix:

1	0
0	−1

Z = [[1,0],[0,−1]]

Qiskit: `qc.z(0)`

Effect: Flips the phase of $|1\rangle$. Leaves $|0\rangle$ alone.

- $Z|0\rangle = |0\rangle$ (unchanged)
- $Z|1\rangle = -|1\rangle$
- $Z|+\rangle = |-\rangle$ (detectable phase flip)
- $Z|-\rangle = |+\rangle$
- $Z^2 = I$ (involution)
- $H \cdot Z \cdot H = X$ (key identity)
- **Rotation:** 180° around z-axis on Bloch sphere
- Eigenstates: $|0\rangle$ (+1) and $|1\rangle$ (−1)

Key Pauli properties (exam favourite)

All three (X, Y, Z) are Hermitian: $G^\dagger = G$

All three are involutions: $G^2 = I$ (apply twice = back to start)

All three are unitary: $GG^\dagger = I$, $G^\dagger = G^{-1}$

Pauli matrices anticommute: $XY = -YX$, $YZ = -ZY$, $ZX = -XZ$

$XYZ = iI$ (product of all three = i times identity)

1.3 Hadamard Gate (H)

Matrix:

1/√2	1/√2
1/√2	−1/√2

H = (1/√2)[[1,1],[1,−1]]

Qiskit: `qc.h(0)`

Effect: Creates / collapses superposition.

- $H|0\rangle = |+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$
- $H|1\rangle = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$
- $H|+\rangle = |0\rangle$
- $H|-\rangle = |1\rangle$
- $H^2 = I$ (applying H twice cancels out)
- $H \cdot X \cdot H = Z$ and $H \cdot Z \cdot H = X$
- **Rotation:** 180° around the X+Z diagonal axis

1.4 S and T Gates (Z-axis rotations)

S and T are both phase gates — they apply a phase factor to the |1⟩ component only, leaving |0⟩ unchanged. They are fractional versions of Z.

S |0> = |0> T |0> = |0>

S |1> = i|1> T |1> = `e^iπ/4 |1>`

S is a rotation about the Z-axis by 90°

T is a rotation about the Z-axis by 45°

SXS† = Y

SYS† = -X

SZS† = Z

Gate	Matrix	Effect on |1⟩	Relation	Qiskit
Z	`[[1,0],[0,-1]]`	phase π (factor of -1)	Z = S^2 = T^4	`qc.z(0)`
S	`[[1,0],[0,i]]`	phase π/2 (factor of i)	S = T^2	`qc.s(0)`
S†	`[[1,0],[0,-i]]`	phase -π/2 (factor of -i)	S† = T^6	`qc.sdg(0)`
T	`[[1,0],[0,e^iπ/4]]`	phase π/4	T = Rz(π/4) up to phase	`qc.t(0)`
T†	`[[1,0],[0,e^-iπ/4]]`	phase -π/4	T† = Rz(-π/4) up to phase	`qc.tdg(0)`

Exam trap — T gate applied to |1⟩

T|1⟩ = e^(iπ/4)|1⟩ → introduces a global phase of π/4

T|0⟩ = |0⟩ (unchanged — the top row of T's matrix is [1,0])

The exam asks: 'which global phase does T introduce to |1⟩ ?' → Answer: π/4

2. Rotation Gates — Rx, Ry, Rz

Rotation gates rotate the qubit state by an arbitrary angle θ around one of the three Bloch sphere axes. Unlike Pauli gates (which rotate by exactly π), rotation gates let you set any angle.

2.1 Rx(θ) — rotation around x-axis

```
Rx(θ) = [[ cos(θ/2),   -i·sin(θ/2) ],
          [ -i·sin(θ/2), cos(θ/2)   ]]

qc.rx(theta, qubit)      # theta in radians
qc.rx(np.pi/2, 0)        # 90-degree rotation
```

- Rx(π) = −iX (equivalent to X up to global phase)
- Rx(π/2)|0⟩ = (|0⟩ − i|1⟩)/√2 → probability 0.5 for both outcomes

2.2 Ry(θ) — rotation around y-axis

```
Ry(θ) = [[ cos(θ/2),  -sin(θ/2) ],
          [ sin(θ/2),   cos(θ/2) ]]

qc.ry(theta, qubit)
qc.ry(np.pi/2, 0)        # rotates |0> to |+> state
```

- Ry(π) = −iY (equivalent to Y up to global phase)
- `Ry(π/2)|0⟩ = (|0⟩ + |1⟩)/√2 = |+⟩` — probability 0.5 each
- Ry has real matrix entries only — no imaginary numbers
- **Exam question type:** 'After qc.ry(np.pi/2, 0), what is the probability of measuring |1⟩ → 0.5

2.3 Rz(θ) — rotation around z-axis

```
Rz(θ) = [[ e^(-iθ/2),    0        ],
          [ 0,           e^(iθ/2)  ]]

qc.rz(theta, qubit)
```

- Rz(π) = −iZ (equivalent to Z up to global phase)
- Rz(π/2) = −iS (equivalent to S up to global phase)
- Rz(π/4) = −iT (equivalent to T up to global phase)
- Note: angle argument is first, qubit index is second — same for all rotation gates

3. Multi-Qubit Gates

Multi-qubit gates act on two or more qubits. They are represented by larger unitary matrices (4×4 for two-qubit gates, 8×8 for three-qubit). The key concept is the control-target structure: a control qubit determines whether the gate is applied to the target qubit.

3.1 CX / CNOT — Controlled-X

The most important two-qubit gate. Flips the target qubit if and only if the control qubit is |1⟩ .

```
qc.cx(control, target)     # control qubit first
qc.cx(0, 1)                # qubit 0 controls qubit 1
```

Input |ctrl⟩ |tgt⟩	Output	What happened
|00⟩	|00⟩	Control is 0 — target unchanged
|01⟩	|01⟩	Control is 0 — target unchanged
|10⟩	|11⟩	Control is 1 — target flipped
|11⟩	|10⟩	Control is 1 — target flipped

- CX is the standard gate for creating Bell states (entanglement)
- Bell state: H on qubit 0, then CX(0,1) on |00⟩ → (|00⟩ + |11⟩)/√2
- $CX^2 = I$ (self-inverse)

3.2 CZ — Controlled-Z

Applies a Z gate to the target if the control is |1⟩ . Symmetric — control and target can be swapped.

```
qc.cz(qubit0, qubit1)      # symmetric — order doesn't matter
```

Input	Output	What happened
|00⟩	|00⟩	No change
|01⟩	|01⟩	No change
|10⟩	|10⟩	No change
|11⟩	-|11⟩	Both |1⟩ — phase flipped

3.3 CCX — Toffoli (Controlled-Controlled-X)

Three-qubit gate. Flips the target qubit only when both control qubits are |1⟩ . This is a universal classical gate (NAND) embedded in quantum computing.

```
qc.ccx(ctrl1, ctrl2, target)
qc.ccx(0, 1, 2)            # qubits 0 and 1 control qubit 2
```

- CCX flips target if and only if ctrl1 = 1 AND ctrl2 = 1
- All other input combinations: target is unchanged
- Classical NAND / Toffoli gate — can implement any classical logic

3.4 SWAP

Exchanges the states of two qubits completely.

```
qc.swap(qubit0, qubit1)
```

Input	Output

|00⟩	|00⟩
|01⟩	|10⟩
|10⟩	|01⟩
|11⟩	|11⟩

- SWAP can be decomposed into three CX gates
- **Qiskit also offers:** `qc.iswap(0,1)` (iSWAP adds a phase factor of i)

4. The Pauli Class and Tensor Products

The Pauli class in qiskit.quantum_info represents multi-qubit Pauli operators as tensor products. This is heavily tested in the exam.

4.1 Importing and using Pauli

```
from qiskit.quantum_info import Pauli

p = Pauli('IZ')           # tensor product I ⊗ Z
print(p.to_matrix())      # gives the 4×4 matrix

p2 = Pauli('XY')          # tensor product X ⊗ Y
p3 = Pauli('ZZZ')         # three-qubit: Z ⊗ Z ⊗ Z
```

4.2 String ordering — critical detail

CRITICAL: Qiskit Pauli string ordering is REVERSED

In Qiskit, Pauli('IZ') means I ⊗Z, NOT Z ⊗I

The rightmost character in the string = qubit 0 (least significant)

The leftmost character = most significant qubit

So 'IZ' → qubit 0 gets Z, qubit 1 gets I

This is the opposite of how you write tensor products in textbooks!

4.3 Tensor product calculation

The tensor product (Kronecker product) of two 2×2 matrices A and B gives a 4×4 matrix:

```
A ⊗ B = [[ A[0,0]*B,  A[0,1]*B ],
         [ A[1,0]*B,  A[1,1]*B ]]

# Example: I ⊗ Z
# I = [[1,0],[0,1]]   Z = [[1,0],[0,-1]]
# I ⊗ Z = [[ 1*Z,  0*Z ],  =  [[ 1, 0, 0, 0 ],
#           [ 0*Z,  1*Z ]]      [  0,-1, 0, 0 ],
#                               [  0, 0, 1, 0 ],
#                               [  0, 0, 0,-1 ]]
```

4.4 Phase prefixes

Pauli strings can include a phase prefix:

```
Pauli('-IZ')     # represents -(I ⊗ Z) — negates the whole matrix
Pauli('iXY')     # represents i*(X ⊗ Y)
```

5. Gate Identities and Relations

Gate identities appear repeatedly in the exam — both directly and embedded in code questions. Memorise all of these.

5.1 Hadamard conjugation identities

Identity	Meaning
H·X·H = Z	H transforms X into Z
H·Z·H = X	H transforms Z into X
H·Y·H = −Y	H transforms Y into −Y
H·H = I	H is its own inverse

Practical use: H·X·H = Z means that qc.h(0); qc.x(0); qc.h(0) is equivalent to qc.z(0).

5.2 Phase gate chain

Gate	Phase on |1⟩	As power of T
T	e^(iπ/4) = π/4	T^1
S	e^(iπ/2) = i = π/2	T^2 = S
Z	e^(iπ) = −1 = π	T^4 = S^2 = Z
T†	e^(−iπ/4) = π/4 back	T^7 (or T†)
S†	e^(−iπ/2) = −i	T^6 (or S†)

5.3 Involution identities ($G^2 = I$)

- $X^2 = I$, $Y^2 = I$, $Z^2 = I$
- $H^2 = I$
- $CX^2 = I$, $CZ^2 = I$, $SWAP^2 = I$
- NOT $S^2 = I$ ($S^2 = Z$, not I)
- NOT $T^2 = I$ ($T^2 = S$, not I)

5.4 Commutation and anticommutation

```
XY = iZ,   YZ = iX,   ZX = iY       # cyclic products
YX = -iZ,  ZY = -iX,  XZ = -iY      # anti-cyclic

# Paulis anticommute with each other:
```

```
XY = -YX,  YZ = -ZY,  ZX = -XZ

# Paulis commute with themselves:
XX = I,  YY = I,  ZZ = I
```

6. Statevectors and Measurement

6.1 Statevector representation

An n-qubit system has 2^n basis states. The statevector is a column vector of 2^n complex amplitudes. Qiskit orders the statevector with qubit 0 as the least significant bit.

```
# 1-qubit: [α₀, α₁]  where α₀ = amplitude of |0⟩, α₁ = amplitude of |1⟩
# 2-qubit: [α₀₀, α₀₁, α₁₀, α₁₁]  for |00⟩, |01⟩, |10⟩, |11⟩
# 3-qubit: 8 elements for |000⟩ through |111⟩

from qiskit.quantum info import Statevector
from qiskit import QuantumCircuit

qc = QuantumCircuit(1)
qc.h(0)
sv = Statevector(qc)
print(sv)   # Statevector([0.707+0.j, 0.707+0.j], dims=(2,))
```

6.2 Probabilities from amplitudes

The probability of measuring a particular state is the squared modulus of its amplitude:

```
P(|x⟩) = |amplitude of |x⟩|²

# Example: state = (|0⟩ + i|1⟩)/√2
# Amplitude of |0⟩: 1/√2  →  P(0) = |1/√2|² = 0.5
# Amplitude of |1⟩: i/√2  →  P(1) = |i/√2|² = |i|²·|1/√2|² = 1·0.5 = 0.5

# Probabilities must sum to 1: Σ|αᵢ|² = 1
```

6.3 Sampling from a Statevector

```
from qiskit.quantum_info import Statevector

# Create a 3-qubit statevector directly
sv = Statevector([0, 0, 0.707, 0, 0, -0.707, 0, 0])
# Non-zero amplitudes at indices 2 (|010⟩) and 5 (|101⟩)

counts = sv.sample_counts(shots=1024)
# Will give ~50% '010' and ~50% '101' — signs don't affect probability
```

Important: Qiskit bit ordering in Statevector

Statevector index n corresponds to the binary representation of n

BUT Qiskit reverses qubit order: index 2 = binary 010 = q2=0, q1=1, q0=0

So Statevector element at index 5 (binary 101) = q2=1, q1=0, q0=1

This is the least-significant-first convention — qubit 0 is the rightmost bit

7. QuantumRegister, ClassicalRegister, and if_test

7.1 Building circuits with registers

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

# Named registers
qr = QuantumRegister(2, name='q')      # 2 qubits named q
cr = ClassicalRegister(2, name='c')    # 2 classical bits named c
qc = QuantumCircuit(qr, cr)

# Access individual qubits by index
q0, q1 = qr[0], qr[1]
c0, c1 = cr[0], cr[1]

# Shorthand (no named registers)
qc = QuantumCircuit(2, 2)    # 2 qubits + 2 classical bits
```

7.2 Measurements

```
qc.measure(qubit, clbit)      # measure one qubit into one classical bit
qc.measure(0, 0)              # qubit 0 → classical bit 0
qc.measure_all()              # measure all qubits (adds classical register)
qc.measure(qr, cr)            # measure whole register
```

7.3 Classical control flow — if_test

Dynamic circuits allow classical control: gates are applied conditionally based on measurement results. This is heavily tested.

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

qubits = QuantumRegister(2)
clbits = ClassicalRegister(2)
qc = QuantumCircuit(qubits, clbits)
(q0, q1) = qubits
(c0, c1) = clbits

qc.h(q0)
qc.measure(q0, c0)

# if c0 == 1: apply H to q1
# else:       apply X to q1
with qc.if_test((c0, 1)) as else_:
    qc.h(q1)
with else_:
    qc.x(q1)

qc.measure(q1, c1)
```

- if_test((clbit, value)) — tests if classical bit equals value
- The else_ context manager handles the else branch

- Condition value 1 is written as 0x1 in the circuit diagram
- while_loop, for_loop, switch_case are also available for dynamic circuits

8. Exam Tips and Common Traps

8.1 High-frequency exam topics in Section 1

1. Pauli tensor products via the Pauli class — Pauli('IZ').to_matrix()
2. Applying Ry(θ) and computing measurement probability from cos/sin
3. Identifying circuit diagrams for if_test / else_ control flow
4. Recognising which statevector corresponds to given non-zero amplitudes
5. T gate global phase: $T|1\rangle$ introduces phase π/4, Gate identities: $H \cdot X \cdot H = Z$, $T^2 = S$, $S^2 = Z$

8.2 The most common mistakes

Traps to avoid
TRAP 1: Reversing rotation gate arguments — qc.ry(qubit, angle) is WRONG. Always: qc.ry(angle, qubit)
TRAP 2: Pauli string order — 'IZ' means $I \otimes Z$ (Z on qubit 0), not $Z \otimes I$
TRAP 3: Assuming $T^2 = I$. It doesn't — $T^2 = S$. Only X, Y, Z, H are involutions.
TRAP 4: Confusing global phase (unobservable) with relative phase (observable)
TRAP 5: Statevector index ordering — index 5 = binary 101 = q2q1q0 = 1,0,1 (LSB first)
TRAP 6: if_test condition value — the diagram shows 0x1 (hex) not just 1

8.3 Quick-reference cheat sheet

Gate	Qiskit	Key effect	Self-inverse?
I	qc.id(0)	Nothing	Yes ($I^2=I$)
X	qc.x(0)	Bit flip $\|0\rangle \leftrightarrow \|1\rangle$	Yes ($X^2=I$)
Y	qc.y(0)	Bit + phase flip	Yes ($Y^2=I$)
Z	qc.z(0)	Phase flip on $\|1\rangle$	Yes ($Z^2=I$)
H	qc.h(0)	Superposition	Yes ($H^2=I$)
S	qc.s(0)	π/2 phase on $\|1\rangle$	No ($S^2=Z$)
T	qc.t(0)	π/4 phase on $\|1\rangle$	No ($T^2=S$)
Rx(θ)	qc.rx(θ,0)	Rotate around x	Only if θ=π
Ry(θ)	qc.ry(θ,0)	Rotate around y	Only if θ=π
Rz(θ)	qc.rz(θ,0)	Rotate around z	Only if θ=π
CX	qc.cx(c,t)	Flip target if ctrl=1	Yes ($CX^2=I$)
CZ	qc.cz(a,b)	Phase flip if both=1	Yes ($CZ^2=I$)

CCX	qc.ccx(c1,c2,t)	Flip target if c1=c2=1	Yes
SWAP	qc.swap(a,b)	Exchange qubit states	Yes

CNOT (control 0, target 1) has the following standard conjugation rules on single-qubit Paulis:

- $X \otimes I \mapsto X \otimes X$
- $I \otimes X \mapsto I \otimes X$
- $Z \otimes I \mapsto Z \otimes I$
- $I \otimes Z \mapsto Z \otimes Z$

These come from the fact that:

- An X on the **control** propagates to the target.
- A Z on the **target** propagates back to the control.
- X on target and Z on control remain unchanged under this conjugation.

So explicitly:

- $\mathrm{CNOT}(X \otimes I)\mathrm{CNOT}^\dagger = X \otimes X$
- $\mathrm{CNOT}(I \otimes X)\mathrm{CNOT}^\dagger = I \otimes X$
- $\mathrm{CNOT}(Z \otimes I)\mathrm{CNOT}^\dagger = Z \otimes I$
- $\mathrm{CNOT}(I \otimes Z)\mathrm{CNOT}^\dagger = Z \otimes Z$

Qiskit Developer Certification

Revision Notes

Operators Module | qiskit.quantum_info

Sources: IBM Quantum Documentation
operators-overview | operator-class

1 Overview of Operator Classes

Qiskit represents quantum operators using classes from the qiskit.quantum_info module. The three main classes are:

Class	Description
SparsePauliOp	Linear combination of Pauli strings — the most commonly used class for quantum observables
Pauli	A single Pauli string with an optional phase coefficient from { +1, i, -1, -i }
Operator	A general linear operator stored as a dense matrix; suitable only for small qubit counts

1.1 Standard Import

```
import numpy as np
from qiskit.quantum_info.operators import Operator, Pauli, SparsePauliOp
```

2 SparsePauliOp

SparsePauliOp represents a linear combination of Pauli strings and is the primary class for defining quantum observables.

2.1 Key Concept

- Stores operators as a weighted sum: $c_1 P_1 + c_2 P_2 + \ldots$ where each P_i is a Pauli string and $c_i \in \mathbb{C}$
- Memory-efficient — only stores non-zero terms (sparse)
- The most common way to represent Hamiltonians and measurement observables

2.2 Initialisation: from_sparse_list

The most flexible constructor — accepts a list of (pauli_string, qubit_indices, coefficient) triplets.

```
# Syntax
SparsePauliOp.from_sparse_list(
    [(pauli_string, qubit_indices, coefficient), ...],
    num_qubits=N
)

# Example
op1 = SparsePauliOp.from_sparse_list(
    [("ZX", [1, 4], 1.0), ("YY", [0, 3], -1 + 1j)], num_qubits=5
)
# Output: SparsePauliOp(['XIIZI', 'IYIIY'], coeffs=[ 1.+0.j, -1.+1.j])
```

Note: qubit_indices map onto positions in the full Pauli string — ordering goes right-to-left (qubit 0 is rightmost).

2.3 Arithmetic Operations

SparsePauliOp supports all standard arithmetic operations:

Operation	Syntax / Example
Addition	op1 + op2 — concatenates Pauli terms
Scalar multiplication	2 * op1 — scales all coefficients
Operator multiplication (composition)	op1 @ op2 — computes product of operators
Tensor product	op1.tensor(op2) — Kronecker product

```
op1 + op2          # Addition — coefficients accumulate
2 * op1            # Scalar multiplication
op1 @ op2          # Operator (matrix) multiplication
op1.tensor(op2)    # Tensor product — doubles the qubit count
```

3 Pauli

The Pauli class represents a single Pauli string with an optional phase coefficient.

3.1 Valid Characters

- Pauli characters: "I", "X", "Y", "Z"
- Phase prefix (optional): "" (none = +1), "i" (= i), "-" (= -1), "-i" (= -i)

3.2 Initialisation

```
op1 = Pauli("iXX")    # Phase = i, string = XX
# Output: Pauli("iXX")
```

3.3 Key Attributes and Methods

Attribute / Method	Description
op.dim	Returns the dimension tuple (d, d) of the matrix — e.g., (4, 4) for 2-qubit Pauli
op.phase	Integer phase exponent — 0=(+1), 1=(i), 2=(-1), 3=(-i)
op.to_matrix()	Returns the dense NumPy matrix representation
adjoint()	Returns the adjoint (Hermitian conjugate) of the Pauli
commutes(other)	Returns True if this Pauli commutes with other
anticommutes(other)	Returns True if this Pauli anti-commutes with other
dot(other)	Computes the dot product (composition) with another Pauli

```
op1 = Pauli("iXX")
print(op1.dim)        # (4, 4)
print(op1.phase)      # 3  (corresponding to phase = i, stored as exponent)
print(op1.to_matrix())
```

Exam tip: phase is stored as an integer exponent of i, not the actual phase value. phase=3 means i^3 = -i, not i itself.

4 Operator — Overview and Basics

The Operator class represents a general linear operator as a dense matrix. Memory scales exponentially with qubit count, so it is only suitable for small numbers of qubits.

4.1 Initialisation from a NumPy Array

```
import numpy as np
```

```
from qiskit.quantum_info import Operator

# 2-qubit Pauli XX operator
XX = Operator(np.array([
    [0, 0, 0, 1],
    [0, 0, 1, 0],
    [0, 1, 0, 0],
    [1, 0, 0, 0],
]))
```

4.2 Key Properties

Property / Method	Description
op.data	Returns the underlying NumPy array (the dense matrix)
op.dim	Returns (input_dim, output_dim) — NOTE: reverse of matrix shape
op.input_dims(qargs)	Returns input dimensions for specified subsystems
op.output_dims(qargs)	Returns output dimensions for specified subsystems
op.is_unitary()	Returns True if the operator is unitary

Important: op.dim returns (input_dim, output_dim) — this is the REVERSE order compared to the shape of the underlying NumPy matrix (which is rows × cols = output × input).

4.3 Automatic Dimension Detection

For 2^N × 2^M matrices, Qiskit automatically infers M-qubit input and N-qubit output dimensions:

```
op = Operator(np.random.rand(2**1, 2**2))   # 2x4 matrix
op.input_dims()    # (2, 2)  — 2 qubits in
op.output_dims()   # (2,)    — 1 qubit out

# Non-power-of-2 matrices stored as single subsystem
op = Operator(np.random.rand(6, 6))
op.input_dims()    # (6,)
op.output_dims()   # (6,)
```

4.4 Manual Dimension Specification

```
# Force input dimension
op = Operator(np.random.rand(2, 4), input_dims=[4])
```

```
op.input_dims()    # (4,)

# Mixed qubit + qutrit system
op = Operator(np.random.rand(6, 6), input_dims=[2, 3], output_dims=[2, 3])
op.input_dims()    # (2, 3)   — qubit on subsystem 0, qutrit on subsystem 1
```

5 Converting Other Classes to Operator

Many Qiskit objects can be directly passed to Operator() for conversion:

Source Class	Notes
Pauli	Converts the Pauli string to its 2^n × 2^n matrix representation
Gate / Instruction	Extracts the unitary matrix of the gate (e.g. CXGate, RXGate)
QuantumCircuit	Performs implicit unitary simulation — computes the full unitary matrix

```
from qiskit.quantum_info import Operator, Pauli
from qiskit.circuit.library import CXGate, RXGate
from qiskit.circuit import QuantumCircuit

# From Pauli
Operator(Pauli("XX"))

# From Gate
Operator(CXGate())
Operator(RXGate(np.pi / 2))   # Parameterized gates work too

# From QuantumCircuit (unitary simulation)
circ = QuantumCircuit(10)
circ.h(0)
for j in range(1, 10): circ.cx(j-1, j)
Operator(circ)   # Returns full 1024x1024 unitary matrix
```

Exam tip: QuantumCircuit → Operator conversion will RAISE an exception if the circuit contains: measure, reset, conditional operations, or any gate without a defined unitary matrix.

6 Using Operators in Circuits

Unitary Operators can be inserted directly into a QuantumCircuit using the append() method.

```
from qiskit.circuit import QuantumCircuit
```

```
from qiskit.quantum_info import Operator, Pauli

XX = Operator(Pauli('XX'))
circ = QuantumCircuit(2, 2)
circ.append(XX, [0, 1])   # Converts Operator → UnitaryGate and inserts it
circ.measure([0, 1], [0, 1])
```

Note: If the Operator is NOT unitary, an exception is raised. Check first with op.is_unitary().

Alternatively, a Pauli object can be appended directly — it becomes a sequence of single-qubit Pauli gates:

```
circ2 = QuantumCircuit(2, 2)
circ2.append(Pauli('XX'), [0, 1])   # Converted to individual X gates
circ2.measure([0, 1], [0, 1])
```

Method	Result in Circuit
circ.append(Operator(Pauli('XX')), [0,1])	Inserts a single UnitaryGate
circ.append(Pauli('XX'), [0, 1])	Inserts individual single-qubit Pauli gates

7 Combining Operators

7.1 Tensor Product — A.tensor(B)

Computes the Kronecker product A ⊗B. Subsystem indexing: B is on subsystem 0, A is on subsystem 1.

```
A = Operator(Pauli('X'))
B = Operator(Pauli('Z'))
A.tensor(B)   # = X ⊗ Z  (B on qubit 0, A on qubit 1)
```

7.2 Tensor Expansion — A.expand(B)

Like tensor product but in reverse order: A.expand(B) = B ⊗A. Now A is on subsystem 0, B is on subsystem 1.

```
A.expand(B)   # = Z ⊗ X  (A on qubit 0, B on qubit 1)
```

Method	Formula	Qubit 0	Qubit 1
A.tensor(B)	A ⊗B	B	A
A.expand(B)	B ⊗A	A	B

7.3 Composition — A.compose(B)

Performs matrix multiplication. A.compose(B) computes B · A (B applied after A):

```
A.compose(B)              # = B·A  (B is applied after A)
A.compose(B, front=True)  # = A·B  (B is applied before A)
```

Mnemonic: Think of compose as 'pipe': A.compose(B) = the operator that first does A, then does B — same as B·A in matrix notation.

7.4 Subsystem Composition — qargs

Compose a smaller operator onto specific subsystems of a larger operator. The smaller operator must always be the argument:

```
# Compose a 2-qubit operator onto qubits [0, 2] of a 3-qubit operator
op = Operator(np.eye(2**3))   # 3-qubit identity
XZ = Operator(Pauli('XZ'))
op.compose(XZ, qargs=[0, 2])              # Apply after op
op.compose(YX, qargs=[0, 2], front=True)  # Apply before op
```

Rule: The smaller operator is always the argument of compose(). Dimensions of the specified subsystems must match.

7.5 Linear Combinations

Standard addition, subtraction, and scalar multiplication are supported:

```
XX = Operator(Pauli('XX'))
YY = Operator(Pauli('YY'))
ZZ = Operator(Pauli('ZZ'))
op = 0.5 * (XX + YY - 3 * ZZ)

op.is_unitary()   # False — linear combinations are generally not unitary
```

Important: tensor(), expand(), and compose() preserve unitarity. Linear combinations (addition, subtraction, scalar multiplication) generally do NOT preserve unitarity.

7.6 Implicit Conversion

All combination methods silently convert numpy arrays and lists to Operator objects:

```
# Pass a plain matrix — it is auto-converted to Operator
Operator(np.eye(2)).compose([[0, 1], [1, 0]])
```

8 Comparing Operators

8.1 Equality — == operator

Checks approximate element-wise equality of the underlying matrices:

```
from qiskit.circuit.library import XGate

Operator(Pauli('X')) == Operator(XGate())     # True

# Operators differing by a global phase are NOT equal
Operator(XGate()) == np.exp(1j * 0.5) * Operator(XGate())  # False
```

Exam tip: The == operator is phase-sensitive. Two operators that differ only by a global phase will return False even if they are physically equivalent.

8.2 Process Fidelity — phase-insensitive comparison

process_fidelity from qiskit.quantum_info compares operators in an information-theoretic way. For unitary operators, it is insensitive to global phase:

```
from qiskit.quantum_info import process_fidelity

op_a = Operator(XGate())
op_b = np.exp(1j * 0.5) * Operator(XGate())  # Differs by global phase

F = process_fidelity(op_a, op_b)
print(F)   # 1.0  — fidelity = 1 means operators are equivalent up to global
phase
```

Comparison Method	Phase-Sensitive?	Use Case
== (equality)	YES — global phase matters	Exact matrix comparison
process_fidelity(a, b)	NO — global phase ignored	Physical equivalence of unitaries

Constraint: process_fidelity is only valid for unitary operators (or completely positive channels). Raises an exception for non-CP inputs.

9 Quick Reference Summary

9.1 SparsePauliOp Cheatsheet

Task	Code
Create from sparse list	SparsePauliOp.from_sparse_list([(str, qubits, coeff)], num_qubits=N)
Addition	op1 + op2
Scalar multiply	c * op

Matrix multiply	op1 @ op2
Tensor product	op1.tensor(op2)

9.2 Pauli Cheatsheet

Task	Code / Value
Create	Pauli("iXX")
Get dimension	op.dim → (4, 4) for 2-qubit
Get phase (exponent)	op.phase → integer 0–3
Dense matrix	op.to_matrix()
Commutation check	op.commutes(other)
Anti-commutation check	op.anticommutes(other)

9.3 Operator Cheatsheet

Task	Code
Create from array	Operator(np.array([[...]]))
From Pauli/Gate/Circuit	Operator(Pauli("XX")) / Operator(CXGate()) / Operator(circ)
Dense matrix data	op.data
Dimensions	op.dim → (input_dim, output_dim)
Unitarity check	op.is_unitary()
Insert into circuit	circ.append(op, qargs)
Tensor product	A.tensor(B) → A⊗B (B on q0, A on q1)
Tensor expand (reverse)	A.expand(B) → B⊗A (A on q0, B on q1)
Compose (B after A)	A.compose(B) → B·A
Compose (B before A)	A.compose(B, front=True) → A·B
Subsystem compose	big.compose(small, qargs=[0,2])
Equality (phase-sensitive)	op_a == op_b
Fidelity (phase-insensitive)	process_fidelity(op_a, op_b)

10 Common Pitfalls & Exam Tips

- SparsePauliOp.from_sparse_list: qubit indices run right-to-left — qubit 0 is the rightmost character in the Pauli string.
- Operator.dim returns (input, output) — opposite order to the NumPy matrix shape (rows × cols).
- Operator class uses dense storage — exponential memory — unsuitable for large qubit counts.
- QuantumCircuit → Operator raises an exception if the circuit has measurements, resets, conditional ops, or gates without matrix definitions.
- A.tensor(B): B lands on qubit 0, A on qubit 1. A.expand(B) is the reverse.
- A.compose(B) = B·A, not A·B. Use front=True to reverse: A.compose(B, front=True) = A·B.
- In subsystem compose, the SMALLER operator must always be the argument.
- tensor/expand/compose preserve unitarity; linear combinations (+ - *) do NOT.
- == comparison is phase-sensitive — operators differing by a global phase return False.
- process_fidelity is phase-insensitive for unitaries — use it to check physical equivalence.
- Pauli.phase stores the phase as an integer exponent of i: 0=(+1), 1=(i), 2=(-1), 3=(-i).

IBM Certified Quantum Computation using Qiskit v2.X

Developer — Associate | Exam C1000-179

Hamiltonians, Pauli Class, SparsePauliOp & PauliList

Complete Study Notes — sourced from official IBM Quantum documentation

Section	Topic
1	Hamiltonians — what they are and why they matter
2	Pauli class — single Pauli operators in Qiskit
3	SparsePauliOp — the standard Hamiltonian representation
4	PauliList — efficient batched Pauli operations
5	Operator arithmetic — +, *, @, ^, simplify
6	Converting between representations
7	Hamiltonians with the Estimator primitive
8	Exam traps and quick-reference cheat sheet

1. Hamiltonians

A Hamiltonian is a Hermitian operator that represents the total energy of a quantum system. In quantum computing it plays two roles: it describes what the system physically does over time (time evolution), and it is used as an observable — something you want to measure the expectation value of.

1.1 Why Hamiltonians appear in quantum computing

- In variational algorithms (VQE, QAOA), the goal is to find the quantum state that minimises the expectation value of a Hamiltonian — this minimum is the ground state energy
- In Hamiltonian simulation, the circuit approximates the time evolution operator e^(−iHt)
- In the Estimator primitive, you pass a Hamiltonian as an observable and get back ⟨ ψ|H|ψ⟩

1.2 The Pauli decomposition

Any Hermitian matrix can be written as a weighted sum of Pauli strings. This is the Pauli decomposition:

```
H = c₀·P₀ + c₁·P₁ + c₂·P₂ + ...    (sum of Pauli strings × coefficients)

# Example: a simple 2-qubit Hamiltonian
H = -1.0·II + 0.3·IZ - 0.3·XI - 0.01·ZY + 0.1·YX

# In Qiskit (SparsePauliOp):
from qiskit.quantum_info import SparsePauliOp
H = SparsePauliOp.from_list([
    ("II", -1.0),
    ("IZ",  0.3),
    ("XI", -0.3),
    ("ZY", -0.01),
    ("YX",  0.1)
])
```

This representation is extremely efficient: instead of storing a $2^n \times 2^n$ dense matrix, you only store the non-zero Pauli terms and their coefficients. For most physical Hamiltonians, the number of terms scales polynomially even for hundreds of qubits.

1.3 Hermitian requirement

Why Hamiltonians must be Hermitian
A Hermitian operator satisfies H† = H (conjugate transpose equals itself)
Hermitian operators always have real eigenvalues — which is essential because energy is real
SparsePauliOp with real coefficients is automatically Hermitian
SparsePauliOp with complex coefficients may NOT be Hermitian — check with .is_unitary() or manually
Pauli strings themselves are always Hermitian: X† = X, Y† = Y, Z† = Z, I† = I

1.4 Expectation value

The expectation value of a Hamiltonian H with respect to a state |ψ⟩ is:

```
⟨H⟩ = ⟨ψ|H|ψ⟩

# For H = Σᵢ cᵢ·Pᵢ, this decomposes as:
⟨H⟩ = Σᵢ cᵢ · ⟨ψ|Pᵢ|ψ⟩

# This is exactly what EstimatorV2 computes — see Section 7
```

2. The Pauli Class

The `Pauli` class from `qiskit.quantum_info` represents a single N-qubit Pauli operator — optionally with a phase coefficient from {+1, i, −1, −i}. It is the atomic building block; SparsePauliOp and PauliList are built on top of it.

2.1 Import and construction

```
from qiskit.quantum_info import Pauli

# Single-qubit Paulis
px = Pauli('X')
py = Pauli('Y')
pz = Pauli('Z')
pi = Pauli('I')

# Multi-qubit Pauli strings (tensor products)
p1 = Pauli('IZ')      # I ⊗ Z  — Z on qubit 0, I on qubit 1
p2 = Pauli('XYZ')     # X ⊗ Y ⊗ Z

# With phase prefix: '', 'i', '-', '-i'
p3 = Pauli('-iXYZ')   # -i · (X ⊗ Y ⊗ Z)
p4 = Pauli('iZZ')     # +i · (Z ⊗ Z)
```

CRITICAL: String ordering is REVERSED from tensor product notation
Pauli('IZ') means I ⊗Z — NOT Z ⊗I
The RIGHTMOST character = qubit 0 (least significant qubit)
The LEFTMOST character = most significant qubit
So 'IZ': qubit 0 gets Z, qubit 1 gets I
Pauli('-iXYZ'): qubit 0 = Z, qubit 1 = Y, qubit 2 = X, phase = -i
This is the OPPOSITE of standard mathematical tensor product notation — a frequent exam trap

2.2 Key attributes

Attribute / method	Returns	Example
`p.to_matrix()`	numpy ndarray — the full matrix	`Pauli('Z').to_matrix() → [[1,0],[0,-1]]`
`p.to_operator()`	Operator object (dense matrix)	useful for small systems
`p.num_qubits`	int — number of qubits	`Pauli('IZ').num_qubits → 2`
`p.phase`	int (0,1,2,3) — phase exponent	0=+1, 1=+i, 2=-1, 3=-i
`p.x`	bool array — X part of symplectic repr	internal representation
`p.z`	bool array — Z part of symplectic repr	internal representation
`p.commutes(other)`	bool — True if [p, other] = 0	`Pauli('X').commutes(Pauli('Z')) → False`
`p.anticommutes(other)`	bool — True if {p,other}=0	`Pauli('X').anticommutes(Pauli('Z')) → True`
`p.adjoint()`	Pauli — conjugate transpose	`Pauli('iX').adjoint() → Pauli('-iX')`
`p.inverse()`	Pauli — same as adjoint for Paulis	`P·P.inverse() = I`
`p.equiv(other)`	bool — equal up to global phase	`Pauli('X').equiv(Pauli('-X')) → True`
`p[i]`	Pauli — single qubit i (drops phase)	`Pauli('XYZ')[0] → Pauli('Z')`

2.3 Symplectic representation

Internally, Qiskit stores Pauli operators in the ZX-convention symplectic representation. Each n-qubit Pauli is encoded as two boolean arrays x[n] and z[n] plus a phase integer. This is what makes PauliList and SparsePauliOp efficient — they store many Paulis as numpy boolean matrices.

```
# The encoding rule for a single-qubit Pauli:
# I = z=False, x=False
# X = z=False, x=True
# Y = z=True,  x=True   (with phase factor)
# Z = z=True,  x=False

# Access symplectic arrays directly:
p = Pauli('XYZ')
print(p.x)    # [True, True, False]  (qubit 0 first)
print(p.z)    # [False, True, True]
```

3. SparsePauliOp

SparsePauliOp is **the standard class for representing quantum observables and Hamiltonians in Qiskit**. It stores a linear combination of Pauli strings: $H = \Sigma_i c_i \cdot P_i$, where the Pauli strings are stored as a PauliList and the coefficients as a complex128 numpy array.

3.1 Construction — five ways

Method 1: from a list of (string, coefficient) tuples

```
from qiskit.quantum_info import SparsePauliOp

H = SparsePauliOp.from_list([
    ("II", -1.0),    # -1.0 * I⊗I
    ("IZ",  0.3),    # +0.3 * I⊗Z
    ("XI", -0.3),    # -0.3 * X⊗I
    ("ZY", -0.01),   # complex coefficient also valid
    ("YX",  0.1)
])
# String ordering: rightmost char = qubit 0 (same as Pauli class)
```

Method 2: from_sparse_list — specify qubit indices explicitly

```
# Most flexible: (pauli_string, qubit_indices, coefficient)
# qubit_indices lists WHICH qubits the non-identity gates act on
op = SparsePauliOp.from_sparse_list([
    ("ZX", [1, 4], 1.0),      # Z on qubit 1, X on qubit 4
    ("YY", [0, 3], -1 + 1j),  # Y on qubit 0, Y on qubit 3
], num_qubits=5)

# IMPORTANT: pauli string length must match number of qubit indices
# 'ZX' has 2 characters, [1, 4] has 2 indices — correct
# Qubits not in the index list get identity I automatically
```

Method 3: from a single string or list of strings (coefficient = 1)

```
# Single string — coefficient defaults to 1+0j
op1 = SparsePauliOp('IZ')

# List of strings — each gets coefficient 1+0j
op2 = SparsePauliOp(['II', 'XZ', 'YY'])

# List of strings with explicit coefficients array
import numpy as np
op3 = SparsePauliOp(['II', 'XZ'], coeffs=np.array([0.5, -0.5]))
```

Method 4: from a dense matrix (from_operator)

```
from qiskit.quantum_info import SparsePauliOp, Operator
import numpy as np

# Decompose any Hermitian matrix into Pauli basis
matrix = np.array([[1, 0], [0, -1]])   # this is Z
op = SparsePauliOp.from_operator(Operator(matrix))
print(op)  # SparsePauliOp(['Z'], coeffs=[1.+0.j])

# WARNING: cost is exponential in qubits — only for small systems
```

Method 5: parameterized SparsePauliOp

```
import numpy as np
from qiskit.circuit import ParameterVector
from qiskit.quantum_info import SparsePauliOp

# Create a Hamiltonian with symbolic parameters
theta = ParameterVector('theta', 2)
H = SparsePauliOp(['II', 'XZ'], coeffs=np.array(theta))

# Bind parameters later
H_bound = H.assign_parameters([0.5, -0.3])
```

3.2 Key attributes and properties

Attribute	Type	Description
op.paulis	PauliList	The Pauli strings component
op.coeffs	np.ndarray (complex128)	The coefficient array
op.num_qubits	int	Number of qubits
op.size	int	Number of terms
op.num_terms	int	Same as .size
op.parameters	ParameterView	Free parameters (if parameterised)
op.dim	tuple	Matrix dimensions (2^n, 2^n)

3.3 Key methods

Simplification — combine duplicate terms, remove zeros

```
# simplify() combines duplicate Pauli strings and drops near-zero terms
op = SparsePauliOp(['X', 'X', 'Z'], coeffs=[0.5, 0.5, 1.0])
simplified = op.simplify()
print(simplified)  # SparsePauliOp(['X', 'Z'], coeffs=[1.+0.j, 1.+0.j])

# Optional tolerances:
op.simplify(atol=1e-8, rtol=1e-5)  # default tolerances

# chop() zeros out real/imaginary parts smaller than tolerance
op.chop(tol=1e-10)
```

Convert to matrix

```
# Dense numpy matrix
matrix = op.to_matrix()         # returns np.ndarray

# Sparse CSR matrix (efficient for large operators)
sparse = op.to_matrix(sparse=True)   # returns scipy.sparse.csr_matrix

# Force single-threaded (for reproducibility in tests)
op.to_matrix(force_serial=True)
```

Convert to label list

```
# to_list() → list of (label_string, coefficient) tuples
```

```
for label, coeff in op.to_list():
    print(f'{coeff:.3f} * {label}')

# More efficient for large operators:
labels_array = op.to_list(array=True)  # pre-allocates numpy array

# Lazy iterator (doesn't materialise all at once):
for label, coeff in op.label_iter():
    pass
```

Sorting

```
# argsort() — return indices that would sort the SparsePauliOp
# Default: sort by Pauli string (lexicographic), then by coefficient
indices = op.argsort()

# Sort by weight (number of non-identity terms) first,
# then lexicographic within each weight group:
indices = op.argsort(weight=True)

# Note: argsort returns indices; apply them with indexing
sorted_op = op[indices]
```

Tensor product

```
op1 = SparsePauliOp('XY')   # 2-qubit
op2 = SparsePauliOp('ZI')   # 2-qubit

# tensor() / ^ operator: op1 ⊗ op2 → 4-qubit operator
result = op1.tensor(op2)     # equivalent to op1 ^ op2
result = op1 ^ op2           # same thing

# expand() is the reverse: op2 ⊗ op1
result = op1.expand(op2)
```

Sum of a list of SparsePauliOps

```
ops = [SparsePauliOp('X'), SparsePauliOp('Z'), SparsePauliOp('Y')]

# SparsePauliOp.sum() — faster than Python's built-in sum()
total = SparsePauliOp.sum(ops)

# Raises QiskitError if list is empty or qubit counts don't match
```

4. PauliList

PauliList is an efficient representation of a **list** of N-qubit Pauli operators. It is the underlying storage for the Pauli strings inside SparsePauliOp (accessible via `op.paulis`). You can use it directly when you need to work with many Paulis as a batch.

4.1 Construction

```
from qiskit.quantum_info import PauliList, Pauli
import numpy as np

# 1. From a list of strings
```

```
pl = PauliList(['II', 'XZ', 'YY', 'ZI'])

# 2. From a list of Pauli objects
pl = PauliList([Pauli('iXI'), Pauli('iZZ')])

# 3. From a single Pauli
pl = PauliList(Pauli('XYZ'))

# 4. From strings with phase prefixes
pl = PauliList(['II', '+ZI', '-iYY'])

# 5. From symplectic arrays
z = np.array([[True, True], [False, False]])
x = np.array([[False, True], [True, False]])
phase = np.array([0, 1])
pl = PauliList.from_symplectic(z, x, phase)
```

4.2 Indexing and iteration

```
pl = PauliList(['II', 'XZ', 'YY', 'ZI'])

# Integer index → returns a single Pauli object
print(pl[0])          # Pauli('II')
print(pl[2])          # Pauli('YY')

# Slice or list index → returns a PauliList
print(pl[1:3])        # PauliList(['XZ', 'YY'])
print(pl[[0, 2]])     # PauliList(['II', 'YY'])

# Iterate like a list
for p in pl:
    print(p)          # each item is a Pauli object

# Lazy label iterator (efficient for large lists)
for label in pl.label_iter():
    print(label)      # string like 'XZ'

# Lazy matrix iterator
for matrix in pl.matrix_iter():
    print(matrix)     # numpy array

# Convert all to matrices at once
matrices = pl.to_matrix()           # dense, shape (n, 2^q, 2^q)
matrices = pl.to_matrix(sparse=True) # sparse CSR matrices

# Convert all to label strings at once
labels = pl.to_labels()             # list of strings
```

4.3 Commutation checking

```
pl = PauliList(['XX', 'YY', 'ZZ', 'XY'])

# commutes(other) — returns boolean array
# True for each Pauli that COMMUTES with 'other'
result = pl.commutes(Pauli('XX'))
# → array([True, True, True, False])  (XY anticommutes with XX)
```

```
# commutes_with_all(others) — True for each Pauli that commutes
# with ALL Paulis in 'others'
result = pl.commutes(PauliList(['XX', 'ZZ']))

# anticommutes(other) — returns boolean array
result = pl.anticommutes(Pauli('XZ'))

# commuting_group() — group all Paulis into commuting sets
# Returns list of PauliLists where each group is mutually commuting
groups = pl.group_commuting()
```

4.4 Sorting

```
from numpy.random import shuffle
from qiskit.quantum_info import PauliList

labels = ['II', 'IX', 'IY', 'IZ', 'XI', 'XX', 'XY', 'XZ',
          'YI', 'YX', 'YY', 'YZ', 'ZI', 'ZX', 'ZY', 'ZZ']
shuffle(labels)
pl = PauliList(labels)

# Default: lexicographic sort
sorted_pl = pl.sort()
# → ['II', 'IX', 'IY', 'IZ', 'XI', 'XX', ...]

# Weight sort: by number of non-identity gates (ascending),
# then lexicographic within each weight
weight_sorted = pl.sort(weight=True)
# → ['II', 'IX', 'IY', 'IZ', 'XI', 'XY', ...]   (weight-1 then weight-2)
```

4.5 Key attributes

Attribute	Returns	Notes
pl.size	int — number of Paulis in list	len(pl) also works
pl.num_qubits	int — qubits per Pauli	all Paulis same size
pl.phase	int array — phase exponents	0=+1, 1=+i, 2=-1, 3=-i
pl.x	bool 2D array — X symplectic part	shape (size, num_qubits)
pl.z	bool 2D array — Z symplectic part	shape (size, num_qubits)
pl.adjoint()	PauliList — conjugate transpose of each	adjoint(P_i) for all i
pl.inverse()	PauliList — inverse of each Pauli	same as adjoint for Paulis

5. Operator Arithmetic

SparsePauliOp supports a full set of arithmetic operators. These are essential for building Hamiltonians programmatically.

5.1 Addition and scalar multiplication

```
op1 = SparsePauliOp(['XIIZI', 'IYIIY'], coeffs=[1.0, -1+1j])
op2 = SparsePauliOp(['ZIIXX', 'IIZZI'], coeffs=[1+2j, -1+1j])

# Addition — concatenates terms (does not simplify automatically)
op3 = op1 + op2
# → SparsePauliOp(['XIIZI', 'IYIIY', 'ZIIXX', 'IIZZI'], ...)

# Scalar multiplication
op4 = 2 * op1
# → SparsePauliOp(['XIIZI', 'IYIIY'], coeffs=[2+0j, -2+2j])

# Best practice: simplify after building
H = (op1 + op2).simplify()
```

5.2 Matrix multiplication (composition)

```
# @ operator: right matrix multiplication (A @ B = A·B)
op5 = op1 @ op2

# compose(): left matrix multiplication by default
# A.compose(B) = A·B  (same as A & B)
op6 = op1.compose(op2)
op6 = op1 & op2           # equivalent

# dot(): right multiplication
# A.dot(B) = B·A  (note the reversal!)
op7 = op1.dot(op2)

# EXAM TIP: A & B == A.compose(B) == B @ A
# compose() is LEFT multiplication, @ is RIGHT multiplication
```

5.3 Tensor product

```
# ^ operator: tensor product
op_tensor = op1 ^ op2        # op1.tensor(op2) = op1 ⊗ op2

# This increases the qubit count: 5-qubit ⊗ 5-qubit = 10-qubit

# expand() is the reverse: op1.expand(op2) = op2 ⊗ op1
```

5.4 Adjoint and conjugate

```
op_adj  = op1.adjoint()    # conjugate transpose: (A†)ᵢⱼ = conj(Aⱼᵢ)
op_conj = op1.conjugate()  # complex conjugate only (no transpose)
op_tr   = op1.transpose()  # transpose only (no conjugate)

# For a Hermitian operator (real coefficients): op == op.adjoint()
```

6. Converting Between Representations

From	To	Method / function

SparsePauliOp	dense matrix (np.ndarray)	op.to_matrix()
SparsePauliOp	sparse matrix (CSR)	op.to_matrix(sparse=True)
SparsePauliOp	Operator object	op.to_operator()
SparsePauliOp	list of (label, coeff)	op.to_list()
SparsePauliOp	PauliList	op.paulis (attribute)
SparsePauliOp	coefficients array	op.coeffs (attribute)
dense matrix	SparsePauliOp	SparsePauliOp.from_operator(Operator(matrix))
PauliList	SparsePauliOp	SparsePauliOp(pauli_list, coeffs=...)
Pauli	matrix	Pauli('XY').to_matrix()
Pauli	Operator	Pauli('XY').to_operator()
PauliList	matrices	pl.to_matrix() or pl.matrix_iter()
PauliList	label strings	pl.to_labels() or pl.label_iter()

```
# Full worked example: build → simplify → convert → inspect
from qiskit.quantum_info import SparsePauliOp
import numpy as np

# 1. Build Hamiltonian
H = SparsePauliOp.from_list([
    ("ZZ", 1.0),
    ("XI", 0.5),
    ("IX", 0.5),
    ("ZZ", -0.2),   # duplicate — will be combined by simplify()
])

# 2. Simplify (combine ZZ terms: 1.0 + -0.2 = 0.8)
H = H.simplify()
print(H)  # SparsePauliOp(['ZZ','XI','IX'], coeffs=[0.8, 0.5, 0.5])

# 3. Convert to dense matrix
matrix = H.to_matrix()
print(matrix.shape)   # (4, 4)  — 2 qubits

# 4. Inspect Pauli terms and coefficients
for label, coeff in H.to_list():
    print(f'{coeff.real:+.2f} * {label}')
# +0.80 * ZZ
# +0.50 * XI
# +0.50 * IX
```

7. Hamiltonians with the Estimator Primitive

The primary use of SparsePauliOp in the Qiskit v2 exam context is as the observable passed to EstimatorV2. The Estimator computes the expectation value $\langle \psi|H|\psi\rangle$ for a given circuit and Hamiltonian.

7.1 EstimatorV2 — basic usage

```
from qiskit.quantum_info import SparsePauliOp
from qiskit.primitives import StatevectorEstimator   # local simulator
from qiskit import QuantumCircuit
import numpy as np

# 1. Define the ansatz circuit
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

# 2. Define the observable (Hamiltonian)
H = SparsePauliOp.from_list([('ZZ', 1.0), ('XI', 0.5), ('IX', 0.5)])

# 3. Run with EstimatorV2
estimator = StatevectorEstimator()
pub = (qc, H)                     # a Primitive Unified Bloc
job = estimator.run([pub])
result = job.result()

# 4. Extract expectation value
expval = result[0].data.evs
print(f'⟨H⟩ = {expval:.4f}')
```

7.2 Multiple observables on the same circuit

```
# You can pass multiple (circuit, observable) pairs
H1 = SparsePauliOp('ZZ')
H2 = SparsePauliOp('XX')

pub1 = (qc, H1)
pub2 = (qc, H2)
job = estimator.run([pub1, pub2])
result = job.result()

ev1 = result[0].data.evs   # ⟨ZZ⟩
ev2 = result[1].data.evs   # ⟨XX⟩
```

7.3 VQE pattern — Hamiltonian as ground state finder

```
from qiskit.quantum_info import SparsePauliOp
from qiskit.primitives import StatevectorEstimator
from qiskit.circuit.library import TwoLocal
from scipy.optimize import minimize
import numpy as np

# Hamiltonian
H = SparsePauliOp.from_list([
    ("II", -1.0), ("IZ", 0.3), ("XI", -0.3)
])

# Parameterized ansatz
ansatz = TwoLocal(2, rotation_blocks=['ry'], entanglement_blocks='cz')

# Cost function
estimator = StatevectorEstimator()
```

```
def cost(params):
    bound = ansatz.assign_parameters(params)
    pub = (bound, H)
    return estimator.run([pub]).result()[0].data.evs

# Minimise
x0 = np.zeros(ansatz.num_parameters)
result = minimize(cost, x0, method='COBYLA')
print('Ground state energy:', result.fun)
```

8. Exam Traps and Quick Reference

8.1 The most tested topics

1. Reading the matrix output of Pauli('IZ').to_matrix() — know what I⊗Z looks like
2. Understanding that string ordering is reversed: 'XY' = X⊗Y, so qubit 0 gets Y
3. SparsePauliOp.from_list vs from_sparse_list — knowing when to use each
4. That + on SparsePauliOp concatenates terms without simplifying — .simplify() must be called
5. compose() vs @ vs dot() — the left/right multiplication distinction
6. How SparsePauliOp is passed to EstimatorV2 as an observable

8.2 Traps to avoid

Exam traps

TRAP 1: String ordering. 'IZ' → I⊗Z → qubit 0=Z, qubit 1=I. NOT Z on qubit 1.

TRAP 2: + does NOT simplify. SparsePauliOp(['X'],coeffs=[1]) + SparsePauliOp(['X'],coeffs=[1]) gives TWO X terms, not one X with coeff 2. Call .simplify().

TRAP 3: compose() vs @. A.compose(B) = A·B (left mult). A @ B = A·B (right mult). A & B = A.compose(B). These are the SAME. But A.dot(B) = B·A — reversed!

TRAP 4: from_sparse_list qubit indices are POSITIONAL, not the full string index. ('ZX', [1,4], 1.0) means Z on qubit 1 and X on qubit 4 of a 5-qubit system.

TRAP 5: Phase in Pauli class. phase attribute returns an integer exponent (0,1,2,3) representing (+1,+i,-1,-i), NOT the phase value directly.

TRAP 6: PauliList.commutes() returns a boolean ARRAY, not a single bool — even if you pass a single Pauli.

8.3 Class hierarchy cheat sheet

Class	Purpose	Typical usage
`Pauli`	`Single N-qubit Pauli string with phase`	`Checking commutation, to matrix()`
`PauliList`	`Efficient list of N-qubit Paulis`	`op.paulis, batch commutation, grouping`

SparsePauliOp	$\Sigma_i\ c_i \cdot P_i$ — Hamiltonian / observable	EstimatorV2, VQE, operator arithmetic
Operator	Dense matrix operator	Small systems only, from operator()

8.4 Import cheat sheet

```
from qiskit.quantum_info import Pauli
from qiskit.quantum_info import PauliList
from qiskit.quantum_info import SparsePauliOp
from qiskit.quantum_info import Operator

# All four are in qiskit.quantum_info
# Also importable from qiskit.quantum_info.operators:
from qiskit.quantum_info.operators import Pauli, PauliList, SparsePauliOp
```

8.5 SparsePauliOp method summary

Method / property	What it does
SparsePauliOp.from_list([...])	Build from list of (str, coeff) tuples
SparsePauliOp.from_sparse_list([...], n)	Build from (str, qubit_indices, coeff) triplets
SparsePauliOp.from_operator(op)	Decompose dense matrix into Pauli basis
SparsePauliOp.sum(ops)	Sum a list of SparsePauliOps efficiently
op.simplify(atol, rtol)	Combine duplicates, remove near-zero terms
op.chop(tol)	Zero out small real/imaginary parts
op.to_matrix(sparse=False)	Convert to numpy array (or scipy CSR)
op.to_list(array=False)	List of (label, coeff) tuples
op.label_iter()	Lazy iterator over (label, coeff)
op.paulis	PauliList of Pauli string components
op.coeffs	numpy complex128 array of coefficients
op.size / op.num_qubits	Number of terms / number of qubits
op + other	Add (concatenates, does NOT simplify)
scalar * op	Scale all coefficients
op @ other	Right matrix multiply: op·other
op & other	Left matrix multiply: op.compose(other)
op ^ other	Tensor product: op $\otimes$ other
op.adjoint()	Conjugate transpose
op.transpose()	Transpose only
op.conjugate()	Complex conjugate only
op.assign_parameters(vals)	Bind ParameterVector values